

Shape of Motion — Algorithm Overview

4D Reconstruction from a Single Video

This document explains the **Shape of Motion** algorithm (4D reconstruction from a single video) as a series of process-flow diagrams. Top-down: the end-to-end pipeline first, then preprocessing, initialization, training, per-step loss computation, adaptive Gaussian control, and the render path. The Mermaid sources live in `diagrams/`; rendered PNGs are alongside them.

1 End-to-end pipeline (`scripts/run_4d.py`)

The mask step is the only manual one. All others run automatically and are individually skippable via `--skip-to`.

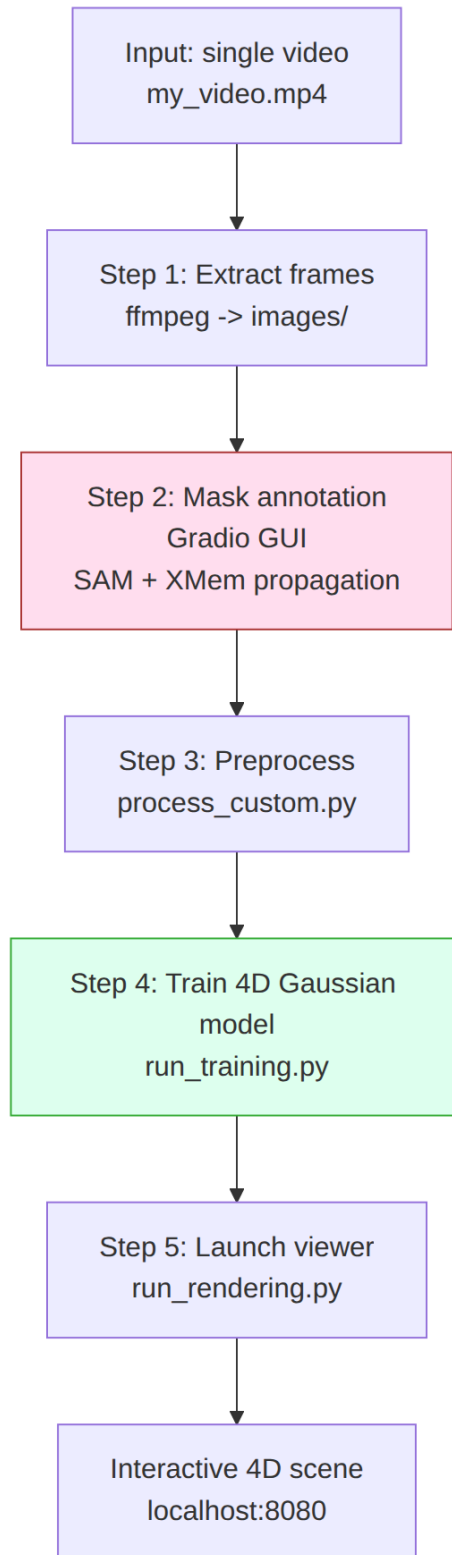


Figure 1: End-to-end pipeline from input video to interactive 4D viewer.

2 Preprocessing (preproc/process_custom.py)

Outputs feed the dataloader: per-frame **RGB + mask + aligned depth + camera** ($K, w2c$) + **2D tracks**.

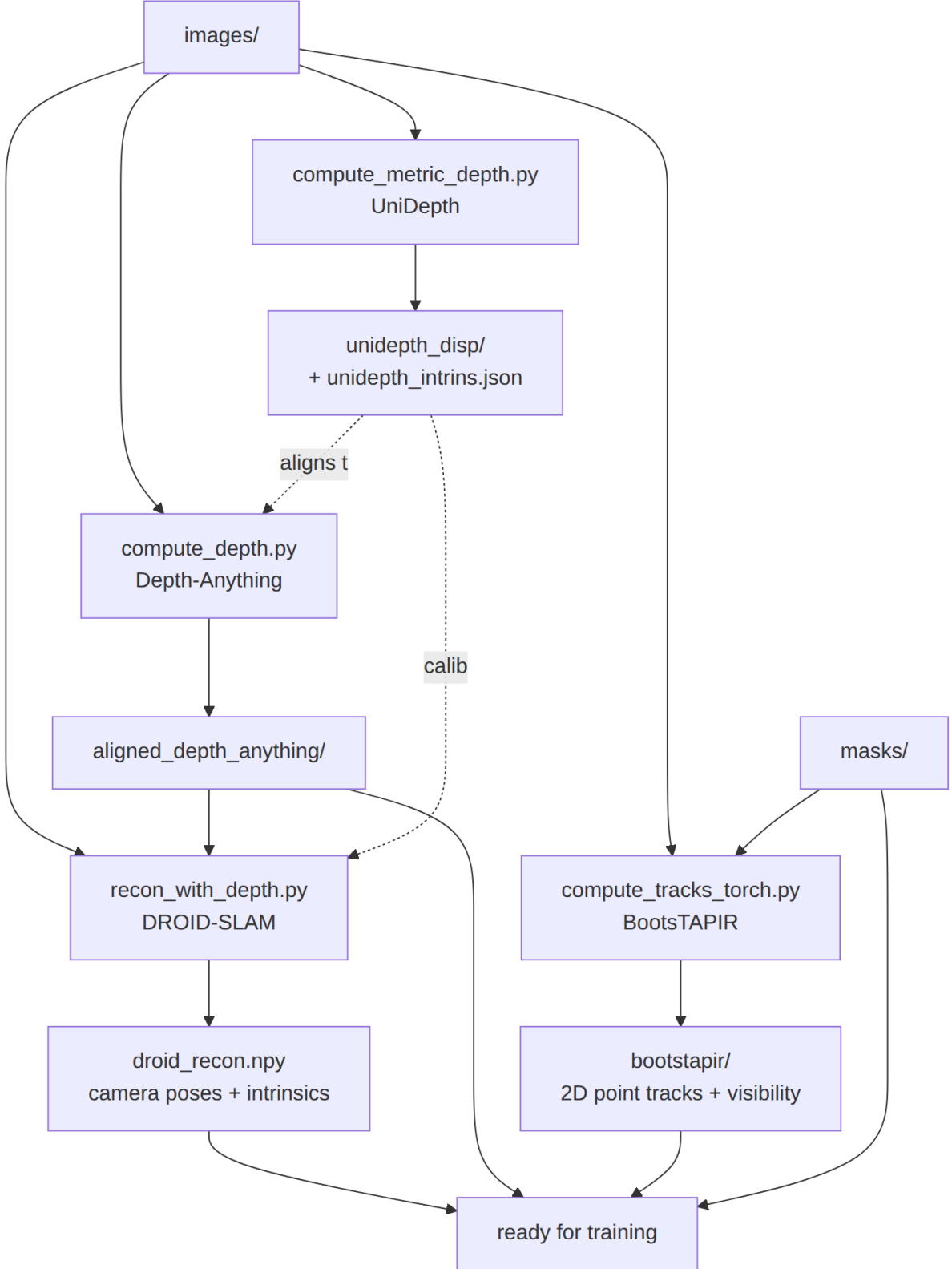


Figure 2: Preprocessing pipeline: UniDepth (metric depth), Depth-Anything (mono depth, aligned to UniDepth), DROID-SLAM (camera poses), and BootsTAPIR (2D tracks).

3 Model initialization (`init_model_from_tracks` $\rightarrow$ `SceneModel`)

Key idea: **foreground motion is parameterized as K shared $\text{SE}(3)$ bases plus per-Gaussian mixing coefficients**, initialized by clustering track trajectories and solving Procrustes per cluster.

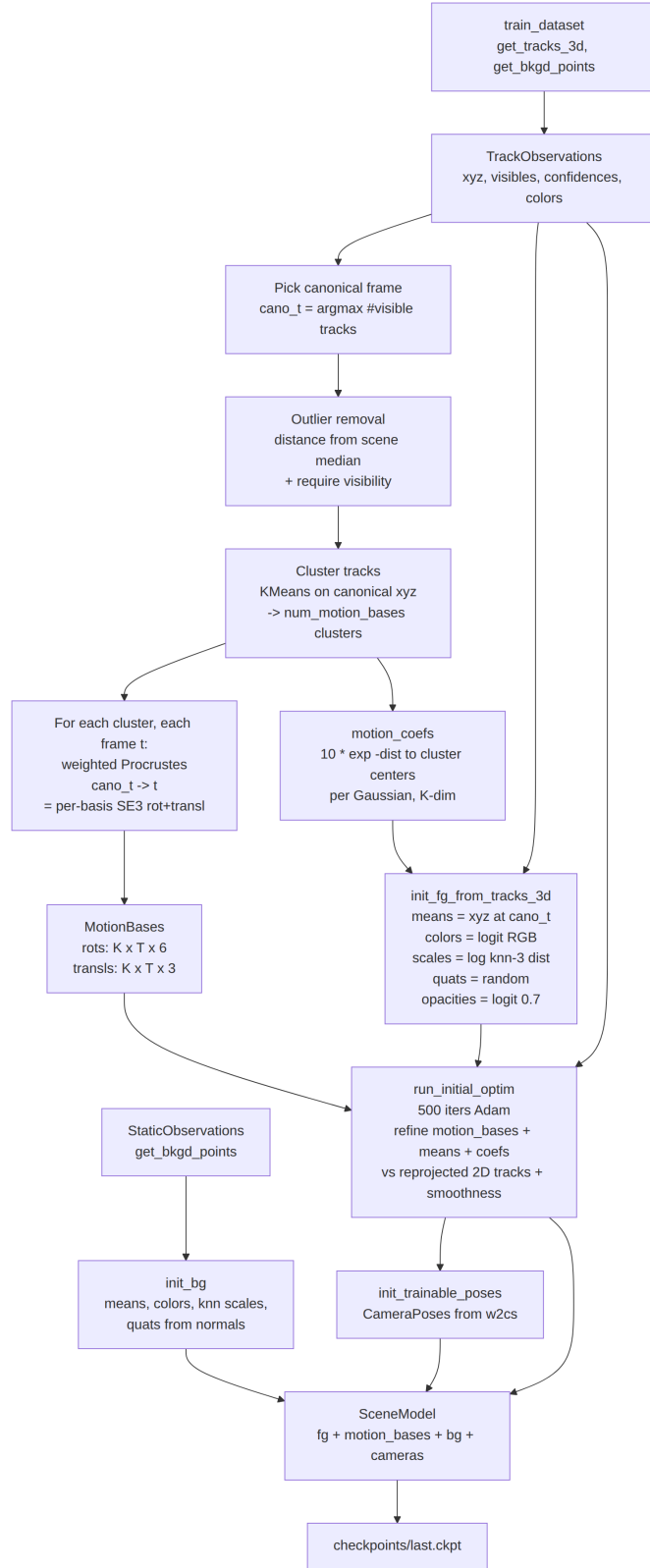


Figure 3: Initialization of foreground Gaussians, motion bases (via weighted Procrustes per cluster), background Gaussians, and trainable camera poses.

4 Training loop (scripts/run_training.py + Trainer)

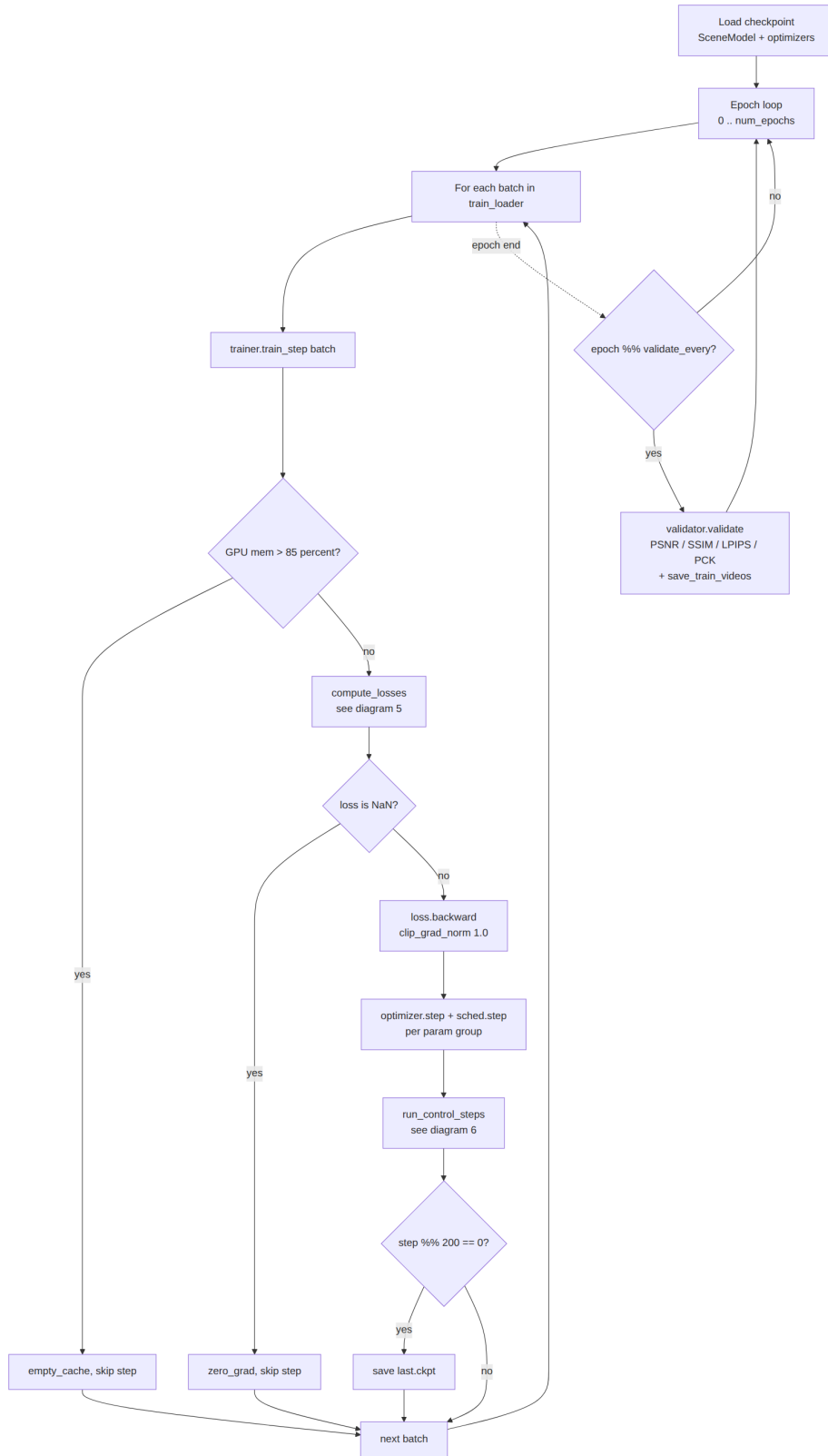


Figure 4: Outer epoch / batch loop with GPU-memory and NaN guards, gradient clipping, periodic checkpointing, and end-of-epoch validation.

5 Per-step loss computation (Trainer.compute_losses)

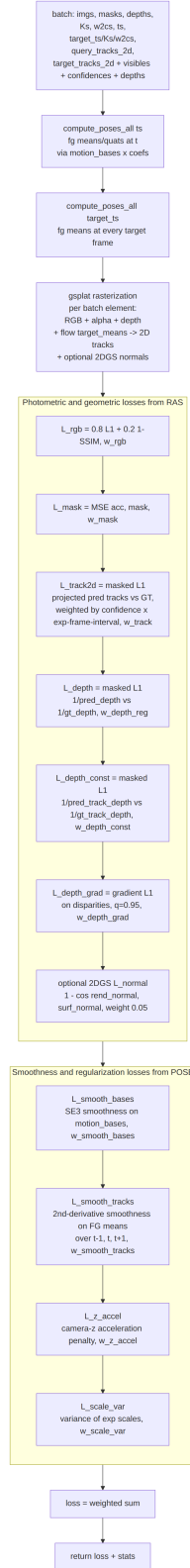


Figure 5: All loss terms: photometric (RGB/SSIM), mask, 2D track reprojection, depth (disparity / track depth / gradient), optional 2DGS normals, and motion smoothness regularizers.

6 Adaptive Gaussian density control (run_control_steps)

These three operations (split / duplicate / cull / opacity-reset) keep the Gaussian count adaptive: dense where the image gradient demands it, sparse elsewhere.

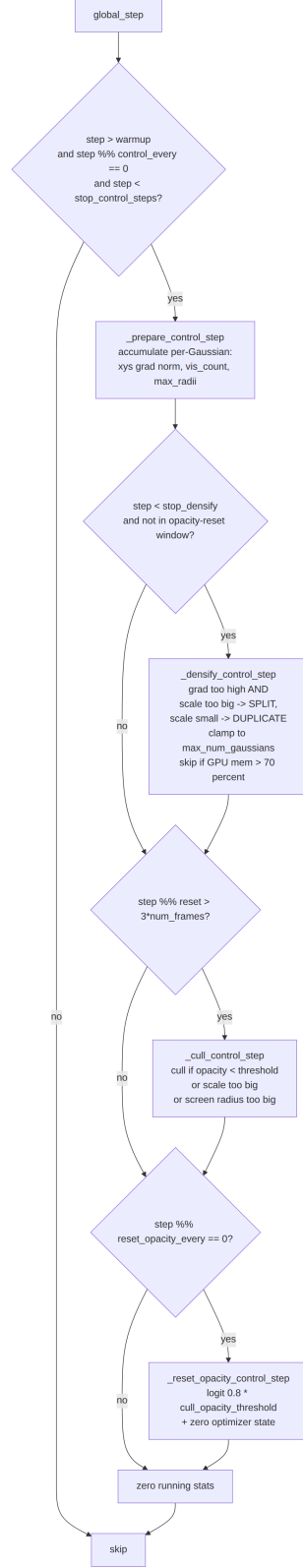


Figure 6: Densify / cull / opacity-reset cycle, gated by warmup, control cadence, and GPU memory pressure.

7 SceneModel render path (used in both training and the viewer)

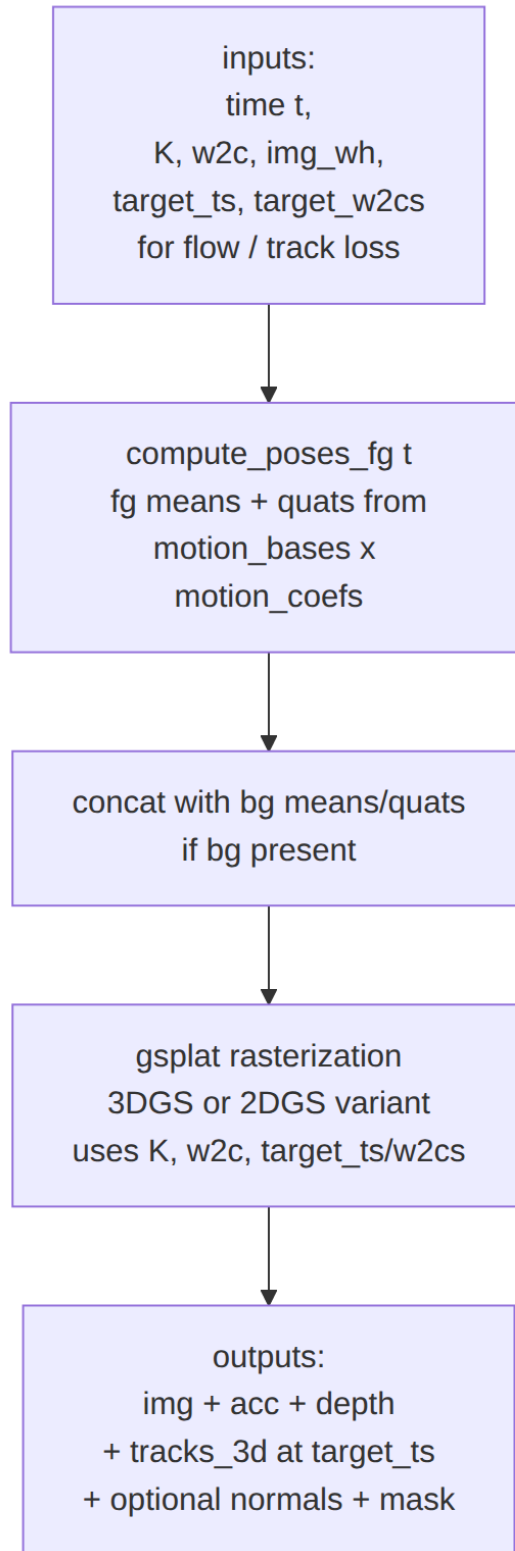


Figure 7: Render path: time-conditioned foreground poses are computed from motion bases $\times$ coefficients, concatenated with static background Gaussians, and rasterized via **gsplat**.

Summary in one sentence

A single video $\rightarrow$ (depth, camera, 2D tracks, masks) $\rightarrow$ cluster the lifted 3D tracks into K motion bases (Procrustes init) $\rightarrow$ represent the dynamic foreground as Gaussians whose poses at any time are mixtures of those K SE(3) bases (plus a static-background Gaussian splat) $\rightarrow$ optimize via `gsplat` rasterization against RGB / mask / depth / 2D-track / smoothness losses, while adaptively splitting, duplicating, culling, and resetting Gaussians.